

Introduction aux tests automatisés dans une chaîne CI/CD

Pourquoi exécuter les tests automatiquement ?

Lorsqu'un développeur travaille sur son poste, il peut exécuter les tests localement avant d'envoyer son code sur le dépôt Git.

Cependant, plusieurs problèmes peuvent apparaître :

- **Le développeur oublie de lancer les tests.**
- **Certains tests sont ignorés volontairement ou involontairement.**
- **Le projet fonctionne sur son poste mais échoue dans un autre environnement.**
- **Une modification casse une fonctionnalité existante sans être détectée.**

Pourquoi exécuter les tests automatiquement ?

Pour réduire ces risques, il est recommandé d'automatiser l'exécution des tests.

- **L'objectif est simple :**

Chaque modification envoyée sur le dépôt doit être vérifiée automatiquement.

Qu'est-ce que la CI ?

CI signifie **Continuous Integration** (Intégration Continue).

L'intégration continue consiste à vérifier automatiquement qu'une modification peut être intégrée au projet sans introduire de régression.

Qu'est-ce que la CI ?

À chaque **Push** ou **Pull Request** :

1. Le code est récupéré.
2. Le projet est compilé.
3. **Les tests sont exécutés.**
4. Les résultats sont analysés.
5. Le développeur est informé du succès ou de l'échec.

Une CI permet de détecter les problèmes quelques secondes après un commit au lieu de plusieurs jours plus tard.

Qu'est-ce que la CD ?

CD signifie **Continuous Delivery** ou Continuous Deployment.

La partie CD intervient **après la validation des tests**.

Exemples :

- Publication d'une application.
- Déploiement sur un serveur.
- Publication d'un conteneur Docker.
- Déploiement dans le Cloud.

Dans notre cas, nous allons uniquement nous intéresser à la partie CI.

Pourquoi utiliser GitHub Actions ?

GitHub Actions est la solution d'automatisation intégrée à GitHub. Elle permet d'exécuter automatiquement des tâches :

- Compilation Maven.
- Exécution des **tests JUnit**.
- Exécution des **tests Mockito**.
- Exécution des **scénarios BDD Cucumber**.
- **Génération des rapports**.
- Déploiement d'applications.

L'automatisation est décrite dans des fichiers YAML stockés directement dans le dépôt Git.

Quels tests allons-nous automatiser ?

Dans notre projet Spring Boot, nous avons déjà mis en place :

Tests TDD

Tests unitaires et d'intégration réalisés avec :

- JUnit 5
- Mockito
- Spring Test

Ces tests vérifient le comportement technique du code.

Quels tests allons-nous automatiser ?

Tests BDD

Tests réalisés avec :

- Cucumber
- Gherkin

Ces tests vérifient le comportement fonctionnel de l'application.

Pourquoi lancer les deux types de tests ?

Les tests TDD et BDD sont complémentaires.

- Les tests TDD répondent à la question :
"Le code fonctionne-t-il correctement ?"
- Les tests BDD répondent à la question :
"L'application se comporte-t-elle comme attendu par l'utilisateur ?"

Une bonne chaîne CI doit vérifier les deux aspects.

Pourquoi générer des rapports ?

Dire qu'un test est réussi ou échoué est utile.

Comprendre ce qui a été testé est encore plus utile.

C'est le rôle des rapports.

Rapport JaCoCo

Le rapport JaCoCo mesure la couverture de code.

Il indique :

- Les classes exécutées.
- Les méthodes exécutées.
- Les lignes couvertes.
- Le pourcentage global de couverture.

Il permet d'identifier les zones du code qui ne sont jamais testées.

Rapport Cucumber

Le rapport Cucumber présente :

- Les Features.
- Les Scenarios.
- Les étapes Given / When / Then.
- Les scénarios réussis.
- Les scénarios en échec.

Il fournit une vue métier du résultat des tests.

Pourquoi utiliser des artefacts ?

Les rapports sont générés **sur le serveur GitHub qui exécute la CI**. Sans artefact, ces fichiers seraient supprimés à la fin de l'exécution. Les **artefacts permettent de conserver les fichiers** produits par la CI afin de les télécharger ultérieurement.

Exemples :

- Rapport JaCoCo HTML
- Rapport Cucumber HTML
- Logs de tests

Les artefacts sont accessibles directement depuis l'exécution GitHub Actions.

Workflow de notre démonstration



À chaque modification, la qualité du code est vérifiée automatiquement et le développeur est informé rapidement du résultat.

Les développeurs téléchargent-ils réellement les rapports JaCoCo et Cucumber ?

Parfois oui, mais ce n'est généralement pas la méthode principale.
Les artefacts sont surtout utilisés :

- **Pour consulter un rapport après un échec.**
- **Pour déboguer un problème rencontré sur la CI.**
- **Pour conserver une trace d'une exécution.**

L'objectif pédagogique des artefacts est excellent car ils permettent de voir concrètement les rapports générés par la chaîne CI.

Comment cela se passe-t-il souvent dans les entreprises ?

Dans les entreprises, les rapports sont généralement centralisés dans des outils spécialisés.

Exemples :

- **SonarQube**
- **SonarCloud**
- **GitLab Quality Reports**
- **Azure DevOps Test Reports**
- **Jenkins + plugins**

Le développeur consulte alors un tableau de bord unique au lieu de télécharger les rapports.

Structure d'un workflow GitHub Actions

Comprendre la structure d'un workflow

GitHub Actions

- Avant d'étudier un workflow concret, il est important de comprendre les principaux éléments qui composent un workflow GitHub Actions.
- Un workflow GitHub Actions est un fichier YAML qui décrit une suite d'opérations exécutées automatiquement par GitHub.

Comprendre la structure d'un workflow GitHub Actions

Les opérations d'un workflow GitHub Actions peuvent permettre :

- **d'exécuter des tests ;**
- de compiler une application ;
- **de générer des rapports ;**
- **de publier des artefacts ;**
- de déployer une application ;
- d'automatiser diverses tâches liées au projet.

Un workflow est généralement composé de plusieurs parties importantes.

Étape 1 — Identifier le workflow

```
name: Mon Workflow
```

Le mot-clé :

```
name
```

permet de donner un nom au workflow. Ce nom sera visible dans l'onglet **Actions** de GitHub.

Exemples :

```
name: Tests Maven  
name: Build Java  
name: Vérification qualité  
name: Déploiement Production
```

Bonne pratique : Choisir un nom explicite permettant d'identifier rapidement le rôle du workflow.

Étape 2 — Déclencher le workflow

on:

Le mot-clé :

on

définit les événements qui déclenchent le workflow.
Sans ce bloc, le workflow ne sera jamais exécuté.

Déclenchement sur un Push

```
on:  
  push:
```

Le workflow se lance lorsqu'un développeur envoie du code sur le dépôt.

Exemple :

```
git push
```

Déclenchement sur certaines branches

```
on:  
  push:  
    branches:  
      - main  
      - develop
```

Le workflow ne sera exécuté que pour les branches indiquées.

Déclenchement sur Pull Request

```
on:  
  pull_request:
```

Le workflow se lance lorsqu'une Pull Request est créée ou mise à jour. Cette pratique est très courante pour vérifier automatiquement le code avant fusion.

Déclenchement manuel

```
on:  
  workflow_dispatch:
```

Permet à un utilisateur de lancer le workflow manuellement depuis GitHub.

Déclenchement conditionnel sur certains fichiers

```
on:  
  push:  
    paths:  
      - "src/**"  
      - "pom.xml"
```

Le workflow ne sera exécuté que si les fichiers concernés ont été modifiés.

Cette technique est particulièrement utile dans les dépôts contenant plusieurs projets.

Étape 3 — Définir les jobs

jobs:

Le mot-clé :

jobs

contient la liste des travaux à exécuter.

Un workflow peut contenir :

- un seul job ;
- plusieurs jobs exécutés en parallèle ;
- plusieurs jobs exécutés dans un ordre précis.

Étape 3 — Définir les jobs

Exemple :

```
jobs:  
  build:  
  test:  
  deploy:
```

Étape 4 – Choisir l'environnement d'exécution

```
runs-on: ubuntu-latest
```

Le mot-clé :

```
runs-on
```

définit le système utilisé pour exécuter le job. Exemples :

```
runs-on: ubuntu-latest  
runs-on: windows-latest  
runs-on: macos-latest
```

Dans les projets Java, Linux est généralement le choix le plus fréquent.

Étape 5 — Définir les étapes du job

steps:

Un job est composé d'une suite d'étapes.

Chaque étape peut :

- exécuter une commande ;
- utiliser une action existante ;
- produire des fichiers ;
- publier des résultats.

Étape 6 – Utiliser une action GitHub

```
uses: actions/checkout@v4
```

Le mot-clé :

```
uses
```

permet d'utiliser une action déjà développée. Une action est un composant réutilisable fourni par GitHub ou par la communauté.

Exemples :

```
uses: actions/checkout@v4  
uses: actions/setup-java@v4  
uses: actions/upload-artifact@v4
```


Paramétrer une action

```
with:
```

Le mot-clé :

```
with
```

permet de transmettre des paramètres à une action.

Exemple :

```
with:  
  java-version: 21
```

Chaque action possède sa propre liste de paramètres.

Étape 7 — Exécuter des commandes

```
run: mvn clean verify
```

Le mot-clé :

```
run
```

permet d'exécuter une commande système. Exemples :

```
run: mvn test  
run: mvn clean verify  
run: java -version  
run: npm install
```

Pour plusieurs commandes :

```
run: |  
    java -version  
    mvn -version
```

Étape 8 — Gérer le répertoire de travail

Dans certains cas, le projet n'est pas situé à la racine du dépôt.

Exemple :

```
repo/  
├─ projet-a/  
├─ projet-b/  
└─ .github/
```

Il est alors possible d'utiliser :

```
working-directory: projet-a
```

ou :

```
defaults:  
  run:  
    working-directory: projet-a
```

pour exécuter les commandes dans le bon dossier.

Étape 9 — Conditions d'exécution

```
if:
```

Le mot-clé :

```
if
```

permet d'exécuter une étape uniquement sous certaines conditions.

Étape 9 — Conditions d'exécution

Exemple :

```
if: always()
```

Cette condition signifie :

Exécuter l'étape même si une étape précédente a échoué.

Cette pratique est très utilisée pour récupérer des rapports ou des logs en cas d'erreur.

Étape 10 – Publier des artefacts

```
uses: actions/upload-artifact@v4
```

Les artefacts permettent de conserver les fichiers générés pendant le workflow. Exemples :

- rapports de tests ;
- rapports JaCoCo ;
- rapports Cucumber ;
- fichiers ZIP ;
- applications compilées.

Sans artefacts, les fichiers générés sont supprimés à la fin de l'exécution.

Étape 11 – Consulter les résultats

Une fois le workflow terminé, GitHub permet :

- de voir le statut du workflow ;
- de consulter les logs ;
- d'identifier les erreurs ;
- de télécharger les artefacts.

Les résultats sont accessibles depuis l'onglet :

Actions

du dépôt GitHub.

Ce qu'il faut retenir

Un workflow GitHub Actions est une description automatisée d'une suite d'opérations. Les éléments les plus importants à connaître sont :

```
name
on
jobs
runs-on
steps
uses
with
run
working-directory
if
upload-artifact
```

Une fois ces mots-clés maîtrisés, il devient beaucoup plus simple de comprendre, modifier ou créer ses propres workflows GitHub Actions.

GitHub Actions : avantages et limites

Pourquoi utiliser GitHub Actions ?

- Intégré directement à GitHub
- Facile à mettre en place
- Automatisation des builds et des tests
- Génération et publication de rapports
- Pas d'infrastructure à installer ou maintenir
- Très adapté aux projets pédagogiques, personnels et aux démonstrations

GitHub Actions : avantages et limites

Limites à connaître

- Quotas d'utilisation sur les comptes gratuits
- Machines d'exécution temporaires (runners éphémères)
- Ressources limitées pour les traitements très lourds
- Les artefacts ne constituent pas un système d'archivage permanent
- Workflows spécifiques à GitHub (migration nécessaire vers GitLab, Jenkins, Azure DevOps, etc.)

Merci pour votre attention

Des questions ?